CSCE361 Voting System

Design Document

Group 9

Project GitHub Repo: https://github.com/Vittoria-Quaranta/Voting-System

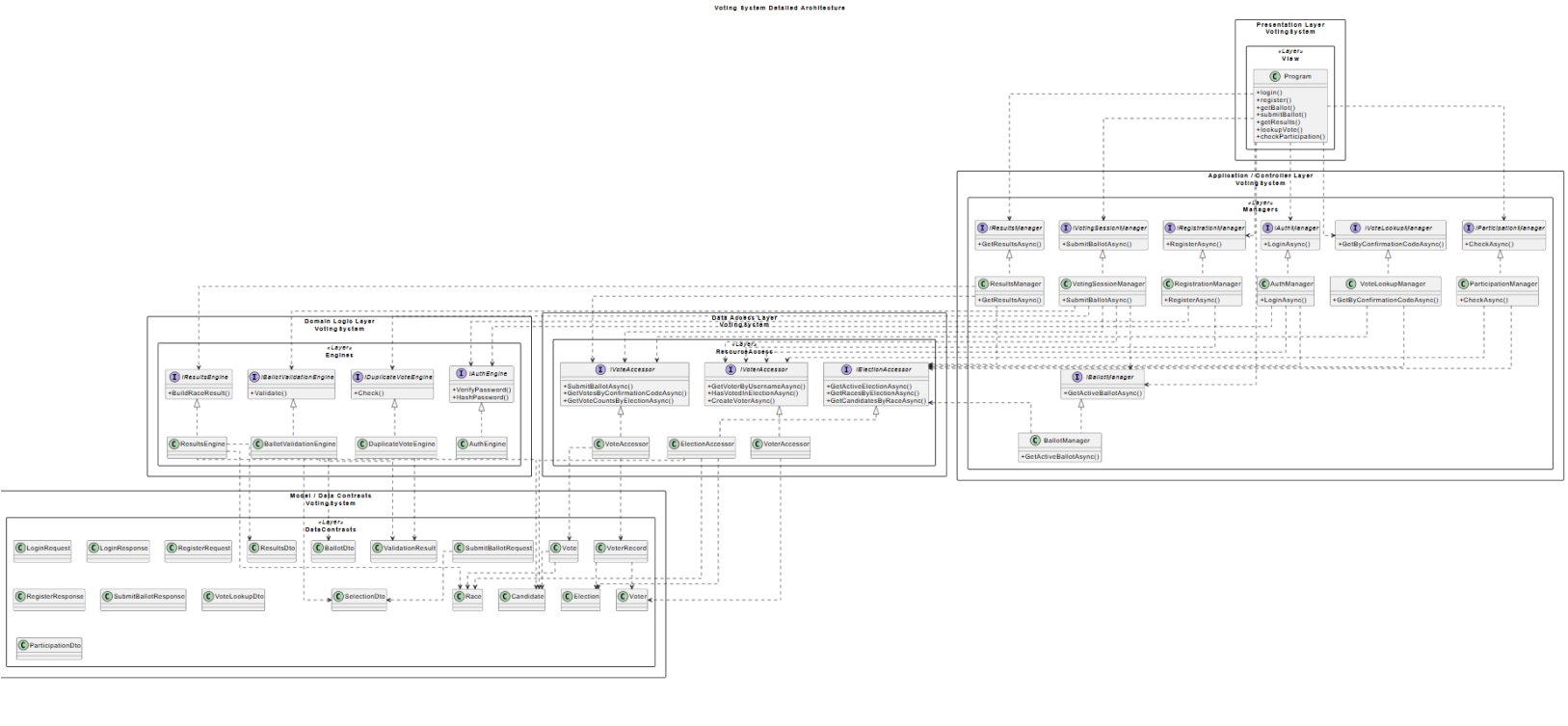

Figure 1: Full Architecture

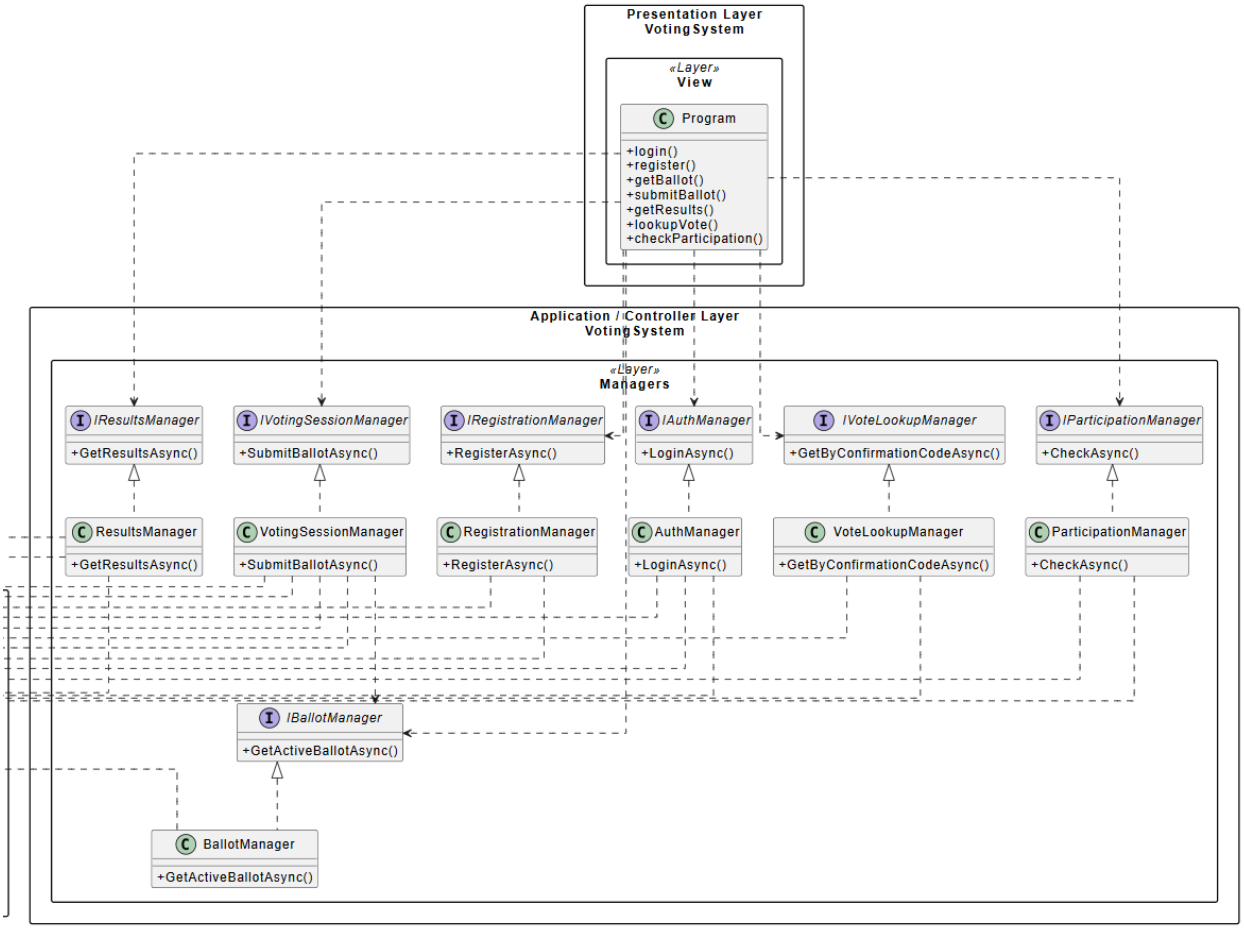

Figure 1.1: Layer (View and Manager)

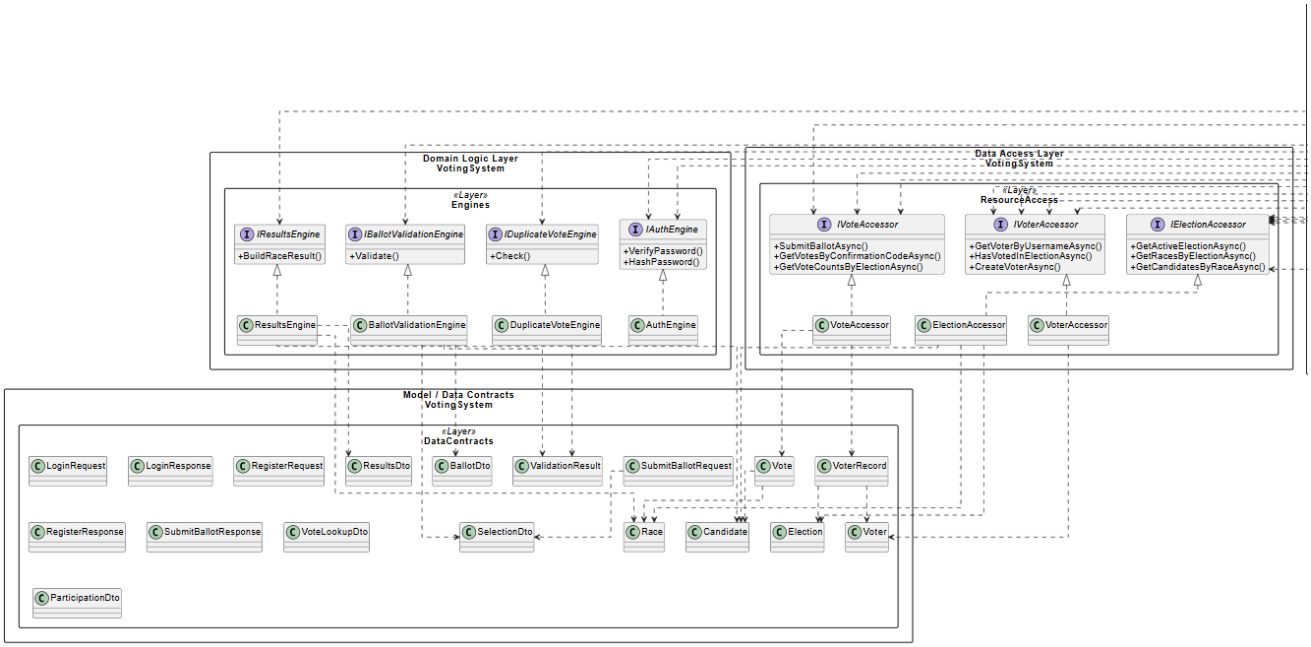

Figure 1.2: Layer (Engines and Resource Access) and Model

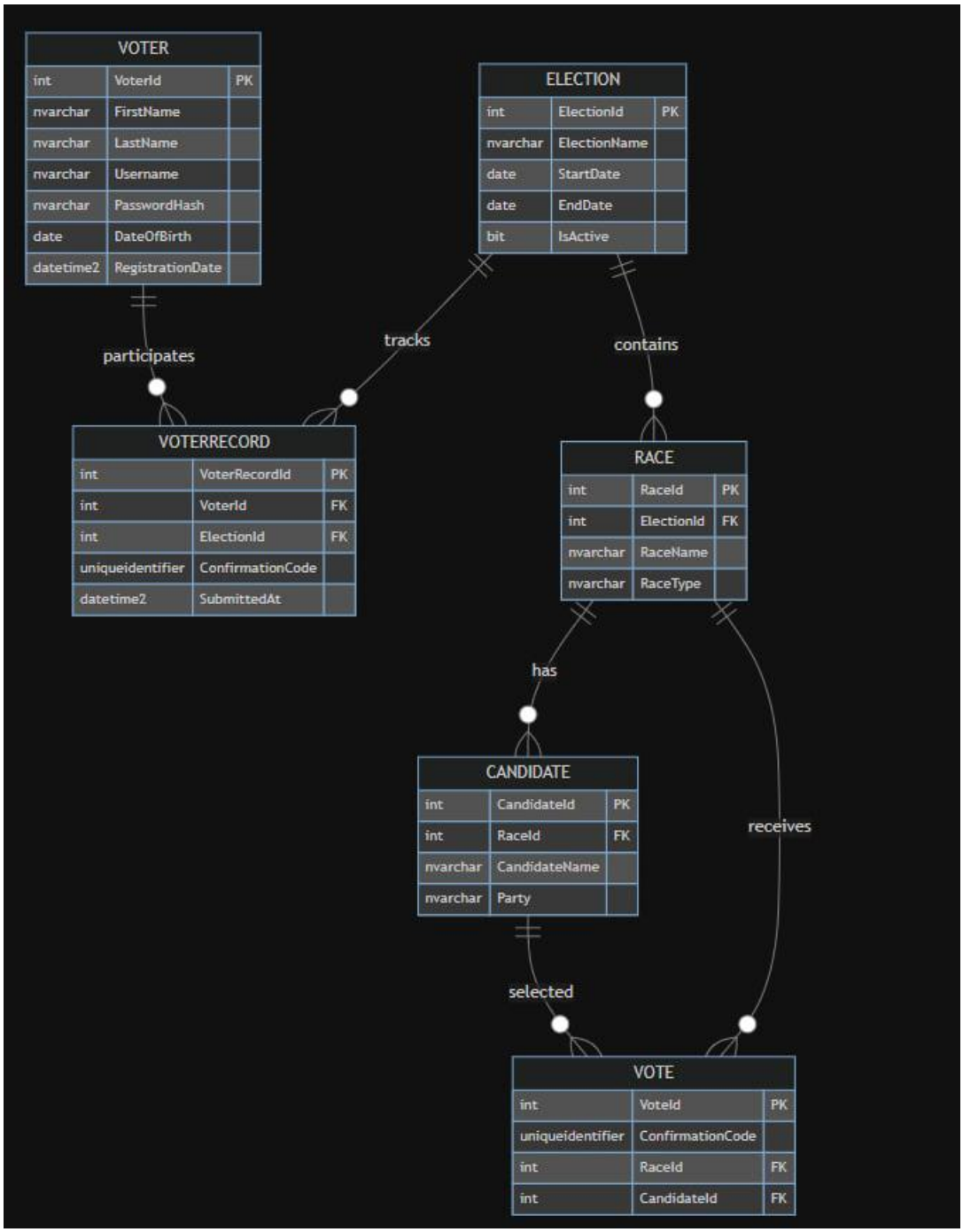

Figure 2: Database

Updated Readme

Voting System

Pacopolis electronic voting system for CSCE 361.

Group Members

- Cameron Adkins
- Yin Po Po Aung
- Vittoria Quaranta
- Sowparnika Sandhya

Client / Use Case Layer

These are the user-facing interactions the system must support:

- Voter Login / Authentication — Voter identifies themselves securely before accessing the ballot.
- Display Ballot — Present the voter with all current elections and issues after authentication.
- Make Selections — Voter selects candidates/choices for each race and issue.
- Review & Edit Selections — Voter can review their full ballot and change selections before submitting.
- Submit Ballot — Final submission locks in the voter's choices.
- View Recorded Vote — A voter can later look up their own vote to confirm it was recorded correctly.
- Third-Party Voter Participation Check — A third party can confirm whether someone voted, but not how they voted.
- Declare Winners — At end of voting day, system calculates and announces winners for each race/issue.

Manager Layer

Orchestrates business logic and workflows:

- Election Manager — Manages the lifecycle of an election: setup, active voting period, and closure.
- Ballot Manager — Assembles the correct ballot for each voter based on active elections.
- Voter Session Manager — Tracks an in-progress voting session; handles review/edit flow before submission.
- Duplicate Vote Prevention — Enforces the rule that a voter may cast a ballot only once per election.
- Results Manager — Tallies votes and determines winners/outcomes at end of voting day.
- Audit/Privacy Manager — Enforces privacy rules: separates voter identity from vote choices in storage and retrieval.

Engine Layer

Core business logic and computation:

- Vote Encryption/Anonymization Engine — Helps prevent a third party from linking a voter to their choices.
- Winner Determination Engine — Implements vote-counting rules for each election type (plurality, majority, yes/no issue, etc.).
- Authentication Engine — Validates voter credentials and identity.

- Ballot Validation Engine — Validates that all required selections are made before submission is allowed.

Resource Access Layer

Data persistence and external integrations:

- Voter Database — Stores voter registration data, credentials, and voting status (voted / not voted).
- Election & Ballot Database — Stores all elections, candidates, issues, and their configurations.
- Vote Record Store — Stores submitted votes in a way that preserves anonymity (decoupled from voter identity).
- Audit Log — Immutable log recording who voted (not how), timestamps, and system events.

Infrastructure / Cross-Cutting

- Multi-Client Access Support — Allow voters to vote from personal computers or shared polling-location machines.
- Role-Based Access Control — Separate access rules for voters vs. admins vs. third-party auditors.
- Election Configuration Interface — Admin UI/tool to define elections, add candidates, set voting windows, and close elections.

Setup

Requirements: .NET 8 SDK, Node.js 18+

Repo layout:

- `frontend/` — React app (Vite)
- `backend/` — C# solution (View, Managers, Engines, ResourceAccess, DataContracts, Tests)

Set up the database connection:

1. Install the **mssql** extension in VS Code
2. Copy the config template:
   - Mac/Linux: `cp backend/VotingSystem.View/appsettings.Development.json.template backend/VotingSystem.View/appsettings.Development.json`
   - Windows: `copy backend\VotingSystem.View\appsettings.Development.json.template backend\VotingSystem.View\appsettings.Development.json`
3. Open `appsettings.Development.json` and replace the `YOUR_*` placeholders with the Azure SQL credentials (get these from the team group chat)
4. Connect to the Azure SQL server in VS Code (`Ctrl+Alt+D` → Add Connection → use the server/credentials from the connection string)
5. Run `backend/Database/Schema.sql` against the `PacopolisVoting` database

> **Note:** Never commit `appsettings.Development.json` — it contains secrets and is gitignored.

Run the backend (development):

```
cd backend
dotnet restore
```

```
dotnet build
ASPNETCORE_ENVIRONMENT=Development dotnet run --project VotingSystem.View
```

> The `ASPNETCORE_ENVIRONMENT=Development` flag is required so the backend reads `appsettings.Development.json` with the real database connection string. Without it, the API will return errors on every request.

Run the frontend (development):

```
cd frontend
npm install
npm run dev
```

Frontend runs on `http://localhost:3000`, proxies `/api` requests to the backend on `http://localhost:5000`.

In dev mode, a **Dev** button appears in the header with tools for testing: reset votes, quick login with test accounts, and page navigation.

Run the frontend (production preview):

```
cd frontend
npm run build
npx vite preview --port 4000
```

Production build on `http://localhost:4000`. Dev tools are stripped out.

Run tests:

```
cd backend
dotnet test
```

Run static analysis:

Backend uses built-in Roslyn analyzers configured via `backend/Directory.Build.props` and `backend/.editorconfig`. Analyzer findings show up as warnings in the build output:

```
cd backend
dotnet build
```

To auto-fix style issues:

```
cd backend
dotnet format
```

Frontend uses ESLint (flat config at `frontend/eslint.config.js`) with React and React Hooks rules:

```
cd frontend
npm run lint
```

Both run in CI; the frontend lint step is blocking.

Test accounts: All share password `husker2026`

Username	Name
tfrazier	Tommie Frazier
ecrouch	Eric Crouch
bberringer	Brook Berringer
jrodgers	Johnny Rodgers
mrozier	Mike Rozier